

COMPSCI 689

Lecture 13: Neural Network Architectures

Part 2

Benjamin M. Marlin

College of Information and Computer Sciences
University of Massachusetts Amherst

Slides by Benjamin M. Marlin (marlin@cs.umass.edu).

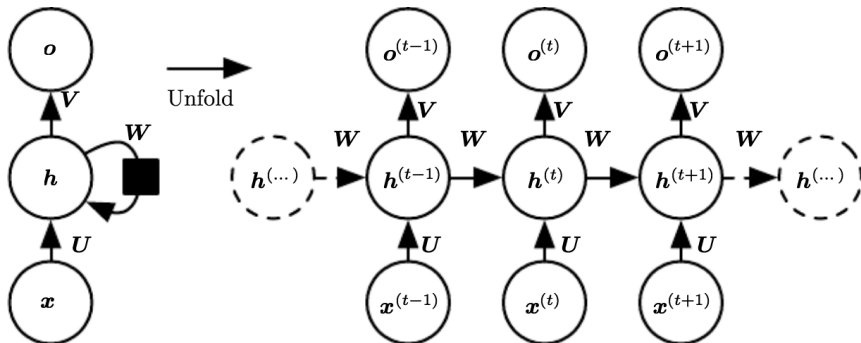
Review

- Basic MLPs have multiple layers of hidden units that are fully connected with distinct weights on each connection.
- Such an architecture does not encode any domain knowledge about the relationship between the inputs and outputs.
- CNNs can be used to model images and encode domain knowledge about spatial invariance.
- We will next consider architectures for sequences, which have different kinds of regularities.

Sequences

- Like with images, sequences often have special structure that can be leveraged by custom network architectures.
- The fundamental idea is that of a *recurrent* architecture that applies the same function to the data (and hidden state) for every position in a sequence.
- This allows for a model to be applied to sequences of any length using a fixed number of parameters.

Basic Recurrent Neural Network: Architecture



Basic Recurrent Neural Network: Prediction

$$\mathbf{h}^{(t)} = \tanh(\mathbf{x}^{(t)}\mathbf{U} + \mathbf{h}^{(t-1)}\mathbf{W})$$
$$o^{(t)} = f(\mathbf{h}^{(t)}\mathbf{V})$$

- For each sequence position t , the RNN computes a linear combination of the input at time t and the hidden state at time $t - 1$ followed by the application of a non-linearity.
- The predicted output at time t is then computed as a linear combination of the hidden state at time t followed by an optional non-linearity.
- The only architectural hyper-parameter in the basic RNN is the size of the hidden representation $\mathbf{h}^{(t)}$.

Basic Recurrent Neural Network: Learning

- To learn the basic recurrent model, it suffices in theory to “unroll” it for a number of steps equal to the length of sequence, and then backprop a prediction losses through the unrolled network.
- Letting L be the prediction loss, $y_n^{(t)}$ be the true output for data case n for position t , and l_n be the length of sequence n , we can apply ERM using the following risk function:

$$R(\theta, \mathcal{D}) = \frac{1}{N} \sum_{n=1}^N \frac{1}{l_n} \sum_{t=1}^{l_n} L(y_n^{(t)}, o_n^{(t)})$$

- Note that there is a choice regarding how to aggregate the loss over sequence positions. The above risk puts equal weight on all sequences regardless of their length.

Basic Recurrent Neural Network: Extras

- Like with the basic MLP, we can get additional control over the basic RNN by regularizing the weights.
- We can also apply techniques such as early stopping or dropout.
- When optimizing the model, we may need resort to multiple of the approaches described for deep MLPs such as momentum, gradient clipping, and the use of adaptive step sizes.

Basic Recurrent Neural Network: Variations

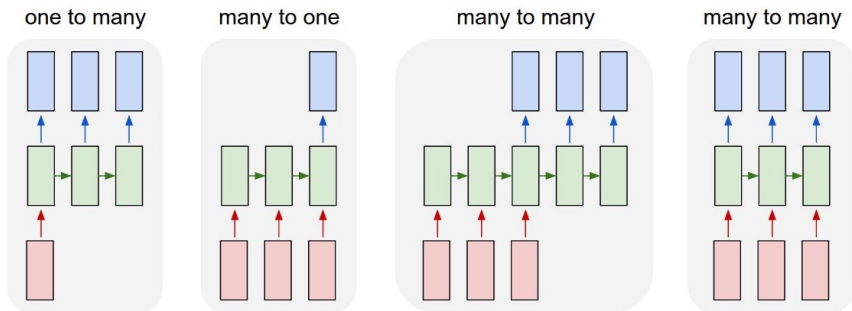
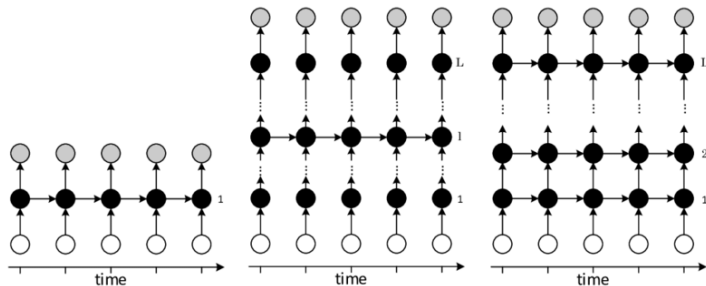


Image Credit: Andrej Karpathy

Deeper RNNs

- There are a variety of ways to increase the depth of an RNN when considering the relationship between the input at position t and the output at position t .
- We can replace any of the linear layers in the basic RNN with multi-layer MLPs that have additional intermediate hidden representation.
- Those additional intermediate hidden representations can themselves have recurrent or non-recurrent structure.

Deeper RNNs

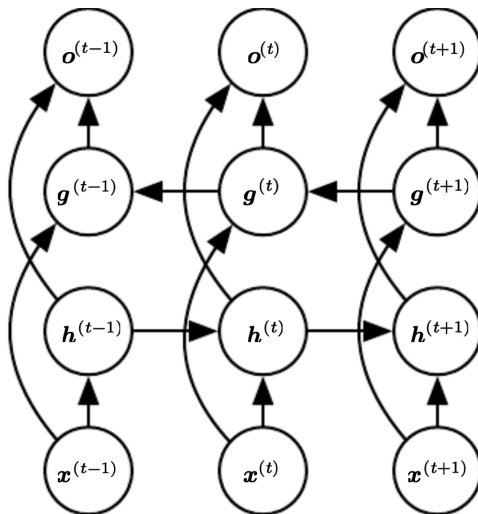


- (Left) Basic RNN
- (Middle) Deep RNN with one recurrent hidden layer.
- (Right) Deep RNN with all hidden layers recurrent.

Bi-directional RNNs

- When an entire input sequence is available at once, there is no particular reason that it should only be processed left to right.
- A bi-directional RNN processes sequences forward and backward, and then computes outputs based on both sets of hidden unit values.
- This requires at least two layers of hidden units.

Bi-directional RNNs



Bi-directional RNN: Prediction

$$\mathbf{h}^{(t)} = \tanh(\mathbf{x}^{(t)}\mathbf{U}_f + \mathbf{h}^{(t-1)}\mathbf{W}_f)$$

$$\mathbf{g}^{(t)} = \tanh(\mathbf{x}^{(t)}\mathbf{U}_b + \mathbf{g}^{(t+1)}\mathbf{W}_b)$$

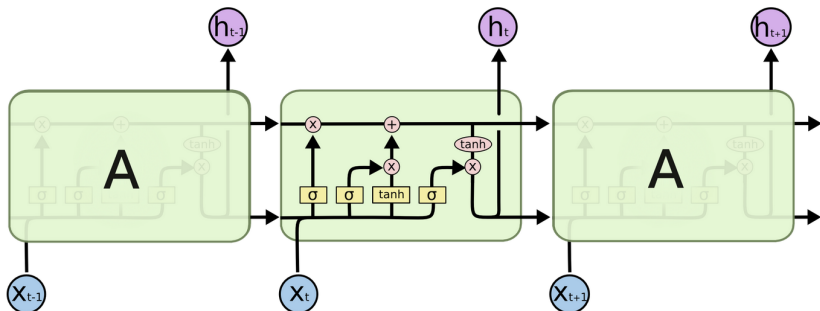
$$o^{(t)} = f(\mathbf{h}^{(t)}\mathbf{V}_f + \mathbf{g}^{(t)}\mathbf{V}_b)$$

- For each sequence position t , we compute the $\mathbf{h}^{(t)}$ hidden unit values only using data from position t and earlier positions in the sequence.
- For each sequence position t , we compute the $\mathbf{g}^{(t)}$ hidden unit values only using data from position t and later positions in the sequence.
- The output $\mathbf{o}^{(t)}$ is computed using $\mathbf{h}^{(t)}$ and $\mathbf{g}^{(t)}$, allowing the entire input sequence to influence the predictions for each t .

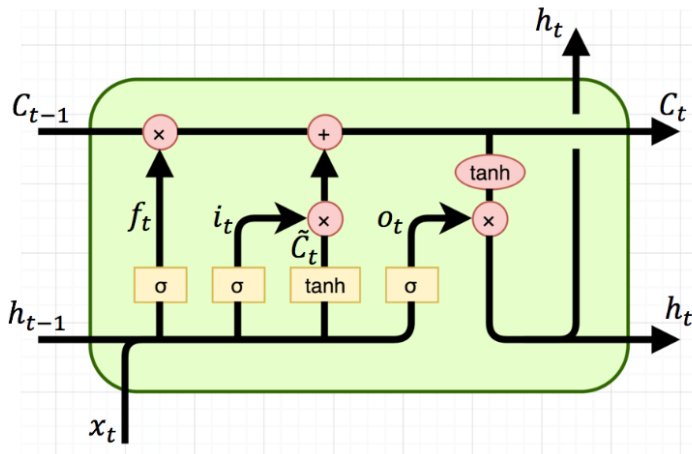
Learning with Long Sequences

- Learning recurrent networks for long sequences, particularly for data with long-range dependencies between inputs and outputs, can be very challenging.
- One major problem is that gradients can explode or vanish depending on the non-linearities used since a recurrent network applied to a long sequence is equivalent to a very deep feed-forward network.
- To deal with this problem, specialized architectures have been developed to improve the propagation of gradient information backward through sequences during learning, and to make it easier to hold on to useful information for long periods of time.

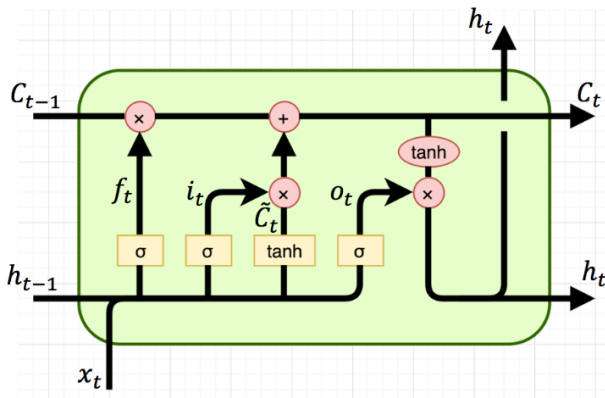
The Long Short-Term Memory Network



The Long Short-Term Memory Cell

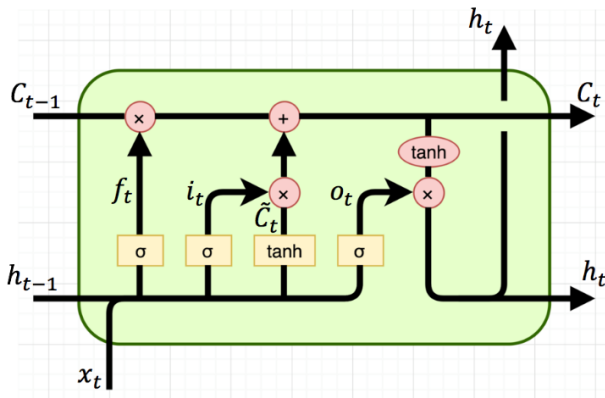


The Long Short-Term Memory Cell



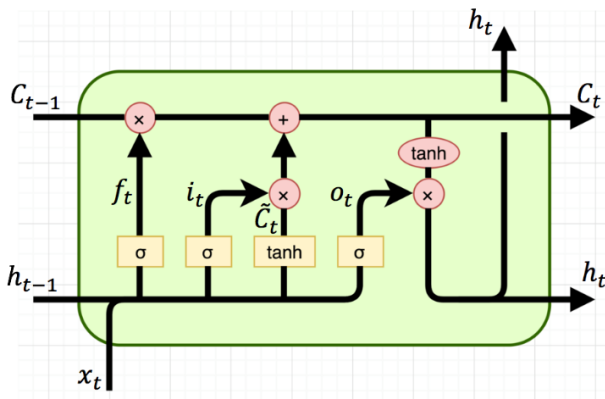
$$\text{Forget gate: } \mathbf{f}^{(t)} = \sigma([\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}] \mathbf{W}_f)$$

The Long Short-Term Memory Cell



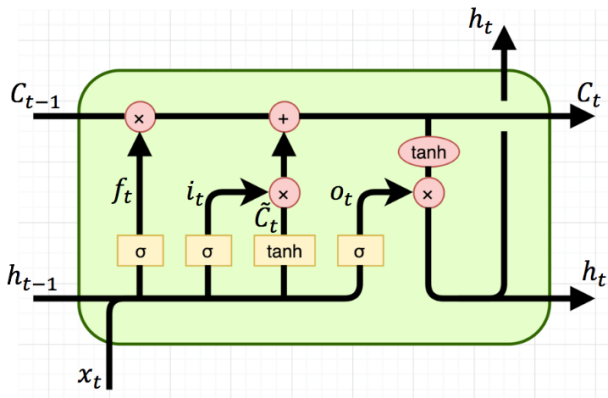
$$\text{Input gate: } \mathbf{i}^{(t)} = \sigma([\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}] \mathbf{W}_i)$$

The Long Short-Term Memory Cell



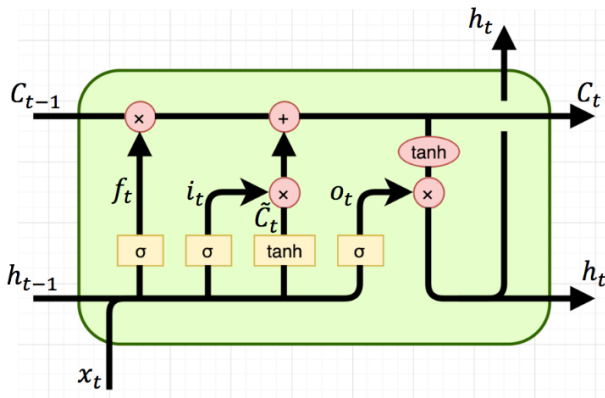
Cell state candidates: $\tilde{\mathbf{c}}^{(t)} = \tanh([\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}] \mathbf{W}_c)$

The Long Short-Term Memory Cell



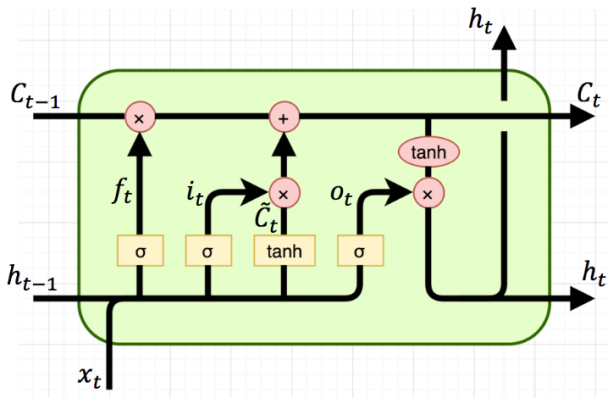
$$\text{New cell state: } \mathbf{c}^{(t)} = \mathbf{f}^{(t)} \odot \mathbf{c}^{(t-1)} + \mathbf{i}^{(t)} \odot \tilde{\mathbf{c}}^{(t)}$$

The Long Short-Term Memory Cell



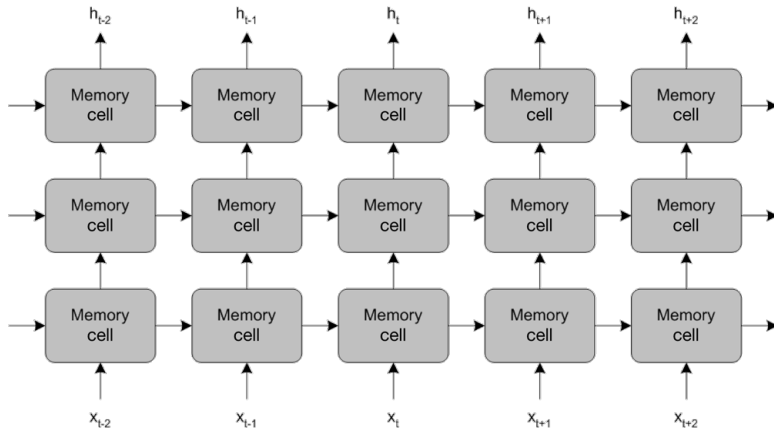
$$\text{Output gate: } \mathbf{o}^{(t)} = \sigma([\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}] \mathbf{W}_c)$$

The Long Short-Term Memory Cell



$$\text{Hidden state: } \mathbf{h}^{(t)} = \mathbf{o}^{(t)} \odot \tanh(\mathbf{c}^{(t)})$$

Deep LSTM Models



Transformers

